

eBPF Governors: QoS-aware Agile Power Management for Latency-critical Datacenter Workloads

Daniel Wong, Associate Professor

Department of Electrical and Computer Engineering, University of California, Riverside

1 Introduction

Power consumption and energy efficiency is now a first-order design constraint in data centers. To save power, servers rely on dynamic power management, such as dynamic voltage-frequency scaling or sleep states. In modern data centers, server power management faces several major challenges: (1) data center workloads are becoming more volatile with shorter and burstier traffic from serverless workloads and ML workloads, (2) existing dynamic power management in Linux can be slow and uncoordinated, and (3) existing Linux dynamic power management subsystems are not quality-of-service (QoS) aware. This can all lead to significant lost power-savings opportunities and poor workload performance.

To highlight the gap between workload volatility and the Linux power management subsystem, Figure 1 shows the power savings achieved by various Linux power management governors when subjected to workload fluctuations oscillating between 50% and 100% utilization with periods of 1ms to 5s. We clearly see as workloads become more volatile, the power savings diminish for existing Linux power management. Schedutil and hardware-managed P-states (HWP) achieves similar power savings. The Userspace governor is augmented to be QoS-aware to take advantage of latency-critical workload’s latency slack, thus achieving better power savings (up to 10%). However, Userspace power savings begins to degrade when workload volatility is less than 100ms, which is now common in data centers [1]. *Essentially, existing Linux power management is not agile and responsive enough to take advantage of fine-grain power savings opportunity in modern data centers.*

1.1 Background

Figure 2 shows an overview of the existing Linux power management subsystems—CPUFreq for frequency scaling (P-states) and CPUIdle for idle states (C-states). CPU power managements are interfaced with the Linux kernel through drivers (e.g. `amd_pstate` for frequency scaling and `intel_idle` for idle states). These low power states are managed by governors, such as `schedutil` governor for CPUFreq ① or `menu` governor for CPUIdle ④. Since these governors reside completely in the driver, they make use of information visible in the kernel, such as the Linux scheduler, to make power management decisions. Therefore, to make QoS-aware P-state selection, many prior works introduce custom power management policies which can observe QoS metrics from the application directly and then utilize the `Userspace` governor ② to initiate changes to P-states [1, 3, 4]. However, this leads to high P-state change overheads due to the `sysfs` interface between the `Userspace` governor and the CPUFreq drivers. For similar reasons, there is a lack of QoS-aware frameworks that interface with CPUIdle due to the microsecond-scale granularity of sleep opportunities [1, 2].

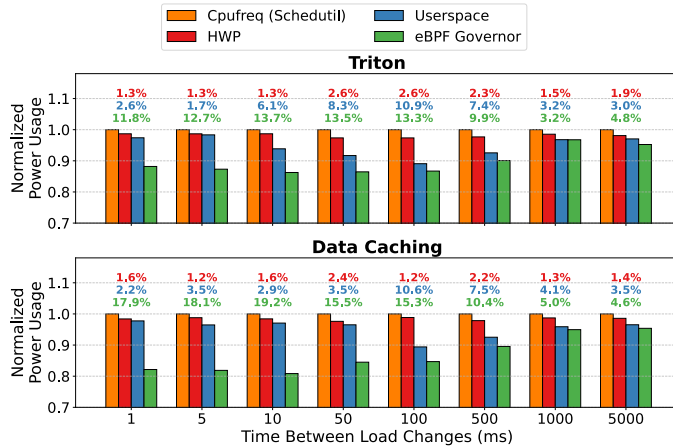


Figure 1: Existing Linux power management governors cannot handle high volatility of modern data center workloads. **eBPF Governors** can sustain power savings while satisfying QoS.

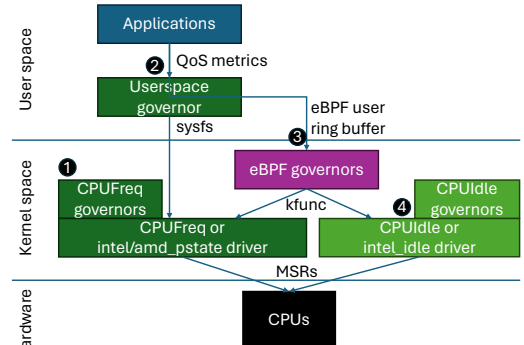


Figure 2: Linux power management subsystem overview with **eBPF Governors** extension.

Related Work: Data center power management of latency-critical workloads is a challenging problem as it requires QoS feedback from applications. This limits many latency-critical QoS-aware power management works to rely on **Userspace** governors [1, 2, 4, 5] and/or limited to evaluation through simulation. **eBPF Governors** would bridge this practicality gap by enabling agile power governors to be QoS-aware. To the best of our knowledge, there are no published works where eBPF is used to control power management states directly. However, there are current trends of moving more and more Linux kernel functionalities into eBPF to provide greater flexibility, such as `sched_ext`.

2 eBPF Governor

The goal of this project is to enable QoS-aware agile dynamic power management by offloading Linux power management governors to eBPF. **eBPF Governors** can enable responsive and agile power management while providing light-weight mechanisms to obtain QoS metrics from latency-critical data center workloads to meet QoS levels. Figure 2 ③ shows how **eBPF Governors** integrates into the existing Linux power management subsystem.

We move power management governors into an eBPF program where users can write custom QoS-aware power management policies. Even though **eBPF Governors** run in the Linux kernel, it can be made QoS-aware through lightweight data sharing mechanisms with userspace through eBPF user ring buffers. This enables **eBPF Governors** to be responsive to application’s workload changes.

eBPF Governors can interface with the **CPUFreq** driver and **CPUIdle** driver through custom eBPF **kfuncs** to facilitate writing to the CPU’s Model Specific Registers (MSRs) which controls the CPU’s power state. For example, we can create **kfuncs** around `wrmsrl_on_cpu` to write to specific MSRs. Another added benefit of **eBPF Governors** is that it can open the door for coordination between both P-states and C-states. Many existing works target only frequency scaling or sleep states [3–5]. Works that coordinates both [1] rely on simulation to demonstrate the benefits due to the decoupled nature of both **CPUFreq** and **CPUIdle** drivers.

Proof-of-concept: To demonstrate the feasibility of **eBPF Governors**, we implemented a proof-of-concept which receives latency metrics from two workloads—**Triton** inference server and **Data Caching** (Memcached). The **eBPF Governors** then calculates a latency slack between the observed tail latency and target tail latency to obtain a desired P-state to achieve the lowest power while still meeting target tail latency. In Figure 1, we can see that **eBPF Governors** can sustain power savings of up to 19%, even under high workload volatility of 1ms. This demonstrates the potential for **eBPF Governors** to achieve agile QoS-aware power management for latency-critical workloads.

Tasks: The goal of this project is to achieve the following: **Task 1:** Extend the proof-of-concept implementation with more sophisticated frequency scaling governor and demonstrate its effectiveness with a larger variety of latency-critical data center workloads. **Task 2:** Extend **eBPF Governors** to be able to manage both frequency (P-states) and bias hardware-managed P-states (HWP). **Task 3:** Extend **eBPF Governors** to replace **CPUIdle** governors and control C-states. **Task 4:** Demonstrate effectiveness of an **eBPF Governors** C-state governor. **Task 5:** Extend and demonstrate coordination of both P-state and C-states.

3 Milestones and Expected Outcomes

September 1, 2025 - Project start.

Milestone 1: December 1, 2025 - (Task 1 and 2) **eBPF Governors** controlling both P-state and biasing HWP. Demonstration with larger variety of workloads. 1st update blog post.

Milestone 2: March 1, 2026 - (Task 3 and 4) Extend **eBPF Governors** to support **CPUIdle** governors and demonstrate effectiveness.

Milestone 3: July 1, 2026 - (Task 5) Extend **eBPF Governors** to coordinate both P-state and C-state management.

August 1, 2026 - 2nd update blog post.

References

- [1] C.-H. Chou, L. N. Bhuyan, and D. Wong. μ DPM: Dynamic power management for the microsecond era. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2019.
- [2] C.-H. Chou, D. Wong, and L. N. Bhuyan. Dynsleep: Fine-grained power management for a latency-critical data center application. In *Proceedings of the 2016 International Symposium on Low Power Electronics and Design, ISLPED ’16*, 2016.
- [3] K. Kaffes, D. Sbirlea, Y. Lin, D. Lo, and C. Kozyrakis. Leveraging application classes to save power in highly-utilized data centers. In *Proceedings of the 11th ACM Symposium on Cloud Computing, SoCC ’20*, 2020.
- [4] H. Kasture, D. B. Bartolini, N. Beckmann, and D. Sanchez. Rubik: Fast analytical power management for latency-critical systems. In *Proceedings of the 48th International Symposium on Microarchitecture, MICRO-48*, 2015.
- [5] X. Zhan, R. Azimi, S. Kanev, D. Brooks, and S. Reda. Carb: A c-state power management arbiter for latency-critical workloads. *IEEE Computer Architecture Letters*, 16(1), 2016.